Deep Learning for Object Detection in Images Development and Evaluation of the YOLOv8 Model Using Ultralytics and Roboflow Libraries

V. V. Kukartsev[1,2], R. A. Ageev[3], A. S. Borodulin[2], A. P. Gantimurov[2], and I. I. Kleshko[1](✉)

[1] Department of Information Economic System, Reshetnev Siberian State University of Science and Technology, 660037 Krasnoyarsk, Russia
ilya-kleshko@mail.ru

[2] Artificial Intelligence Technology Scientific and Education Center, Bauman Moscow State Technical University, 105005 Moscow, Russia

[3] Faculty of Computer Science, HSE University, Moscow, Russia

Abstract. This paper presents a study of the application of the YOLOv8 model for object detection in images using Ultralytics and Roboflow libraries. Deep learning in computer vision is one of the most actively developing areas in modern technologies, allowing to automate the processes of analysis and recognition of objects in images. The YOLO (You Only Look Once) model is one of the most effective and widely used methods for real-time object detection. The study covers the main steps of working with the YOLOv8 model, from installing and configuring the necessary libraries and development environment, to training the model on the provided data and evaluating its performance. We present a detailed analysis of the training process, including selection of optimal hyperparameters, tuning of training parameters and data augmentation.

1 Introduction

In today's world, computer vision is becoming an increasingly important and promising area of research and development. It finds applications in various fields, ranging from industrial automation and medical diagnostic systems to video surveillance systems and autonomous cars. In the context of this project, we will focus on developing a neural network for computer vision using the Python programming language.

The goal of this project is to create a neural network that can automatically analyse images and identify different elements in them, such as empty areas, windows, backgrounds, and overall occupancy. This represents an image segmentation task that has a wide range of practical applications including warehouse inventory management, traffic analysis in urban environments, and more.

The image segmentation process is complex, requiring analysing large amounts of data and training machine learning models on it. In this project, we will use deep learning based on convolutional neural networks to achieve high accuracy in identifying objects of interest in images [1–3].

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2024
R. Silhavy and P. Silhavy (Eds.): CSOC 2024, LNNS 1118, pp. 629–637, 2024.
https://doi.org/10.1007/978-3-031-70285-3_48

Our project represents an important step towards the development of computer vision systems that can adapt to different environments and tasks. We hope that the results of our research and development will not only expand our understanding of computer vision, but also find practical applications in the real world.

2 Installing and Testing the Library

We install and test the Ultralytics library, specialised for the development of computer vision algorithms using deep learning methods.

To start working with the Ultralytics library, it is installed using the Python package manager - pip. This allows you to quickly and conveniently add the library to the development environment. In our case, a specific version of the library is specified for installation.

Using a method to clean up the output is an important step that helps to keep the code output clean and readable. After clearing the output of previous results, new results and output can be analysed more conveniently.

After installing the library, you should check if the library is working correctly. For this purpose, a special method is called that performs a set of test checks. These checks include not only the presence and availability of the library, but also the integrity of the necessary dependencies and resources required for its functioning.

After performing these operations, you can be sure that the Ultralytics library is successfully installed and ready to be used for developing computer vision algorithms.

Ultralytics: Imports the YOLO class from the Ultralytics library. YOLO (You Only Look Once) is an object detection algorithm that can simultaneously detect and classify objects in an image. This class will be used to create and train a neural network model for detecting objects in images.

IPython.display: The display and Image functions from the IPython.display library are imported. They will be used to display images in Jupyter Notepad.

os: The os module for working with the operating system is imported. It will be used to get the current working directory.

The HOME variable is defined as the current working directory where the script is running. This can be useful for working with files and file paths in the project. The value of the HOME variable is printed to ensure that it is defined correctly.

Once the above operations have been performed, the required libraries are imported and the working directory is set, allowing you to continue working with the project.

The Roboflow library is installed using the pip command. The Roboflow library provides tools for working with computer vision data, including loading, augmentation and dataset management. Installation is done using the “magic” command !pip, which allows you to run shell commands directly from the Jupyter Notebook environment.

Next, the Roboflow class is imported from the roboflow library. Then an rf object of the Roboflow class is created, passing an API key for authentication as a parameter. This key allows you to access your Roboflow account and use its functionality.

Once the rf object is created, the project and data version is selected in the Roboflow workspace. To do this, the workspace(), project(), and version() methods are used to

specify the required parameters (e.g., the name of the workspace, project, and dataset version).

The download() method is then called to download the selected dataset version using the YOLOv8 model. The data is downloaded from the selected project version to Roboflow and stored in the dataset variable.

After performing the above operations, the data has been successfully downloaded and is ready to be used for training the neural network model [4, 5].

3 Training, Validation and Prediction Using the YOLOv8 Model

The presented code section performs training, validation and prediction operations using the YOLOv8 model to detect objects in images.

Model definition: An instance of the YOLO model is created specifying the yolov8s model configuration.

Training the model: The train() method of the model is called, where the training parameters such as data (data), task type (task), batch size (batch), number of epochs (epochs), optimiser (optimizer), image size (imgsz) and others are specified. The model is trained on the provided data according to the specified parameters.

Model training is a key stage in the development of computer vision system. This stage allows the model to learn the data and adapt to the specific object detection task.

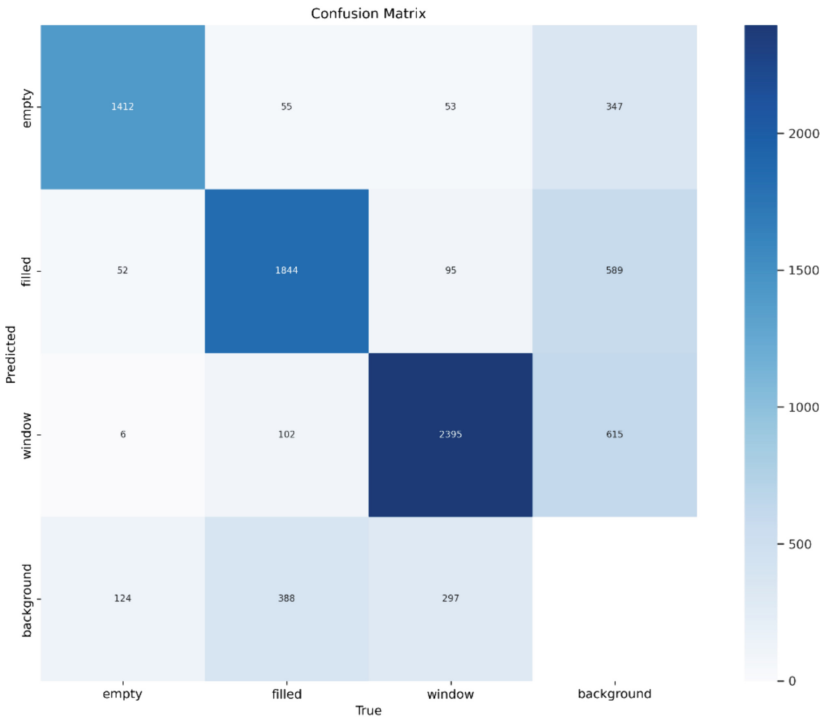

Fig. 1. Confusion matrix

Model validation: a new instance of the YOLO model is created with a path to the best weights (best.pt) that were stored during training.

Model validation is an important step to evaluate the quality of the model and identify problems that may require further tuning or training.

Model validation: The val() method of the model is called for validation. The parameters include the task type (task), validation data (data), IoU threshold (iou), maximum number of detections (max_det), confidence level (conf) and image size (imgsz). This allows us to evaluate the quality of the model performance on the delayed sample [6–10] (Figs. 1, 2).

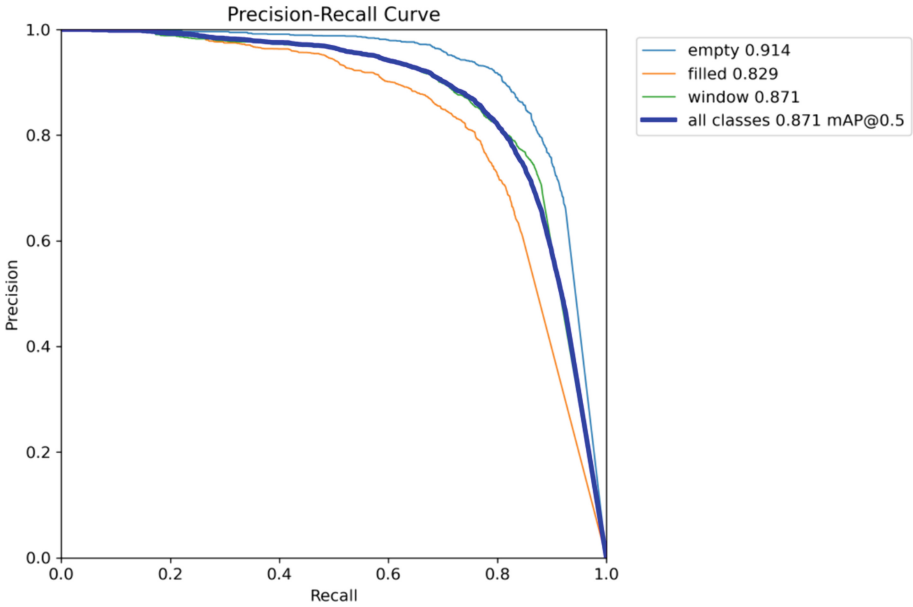

Fig. 2. Precision-Recall Curve

Object prediction: The predict() method of the model is called to predict objects in a particular image.

predict objects in a particular image. The parameters include the source of the image (source) and the parameter to save the results (save and save_txt). The model processes the image and predicts objects, then can save the results in image files and text files with coordinates and classes of the predicted objects. This section of code allows for training, validate and use the trained YOLOv8 model to detect objects in images. Predicting objects using the trained model allows us to put the model into practice and evaluate its performance on real data.

In addition to YOLOv8, there are also other advanced models for object detection in images. For example, Faster R-CNN (Region-based Convolutional Neural Networks) and SSD (Single Shot MultiBox Detector) are also widely used models for detecting objects. Faster R-CNN is characterized by a more complex architecture that includes

two networks: the Region Proposal Network (RPN) for proposing regions of interest and the network for classifying and classifying regions of interest regions of interest and a network for classifying and accurately localizing objects in the proposed areas. SSD, on the other hand, utilizes a single convolutional network to simultaneously predict the class and bounding rectangle for each image region.

Also worth mentioning is EfficientDet, which is a is a family of models with high efficiency and accuracy. EfficientDet combines efficient architecture and object scaling techniques to achieve a better balance between accuracy and computational efficiency.

The choice between the different models depends on the specific project requirements, such as accuracy, processing speed, and available computational resources [11, 12].

The results of the work are presented in Figs. 3–4.

Fig. 3. Original image

Fig. 4. Processed image

4 Evaluation of Results

Model evaluation is an important step in the process of developing a computer vision system. Various metrics such as object detection accuracy, object classification accuracy, and image processing speed are used for this purpose. Conducting an evaluation helps to determine the effectiveness of the model in solving a particular problem and identify possible improvements.

In the case of the YOLOv8 model, evaluation may include analyzing the object detection accuracy expressed, for example, through the Intersection over Union (IoU) metric, which evaluates the overlap of the predicted area with the true label, and the Mean Average Precision (mAP) metric, which considers the accuracy and completeness of object detection for different classes. In addition, model evaluation may also include comparing its speed to other models such as Faster R-CNN, SSD, or EfficientDet to determine the advantages and disadvantages of each model in the context of a particular application [13–15].

Conducting a comprehensive evaluation of a model helps developers make informed decisions about improving, tuning, and optimizing it for specific computer vision applications.

5 Improving the Model

Based on the results of the evaluation, further improvements can be made to the model. This may include changing the model architecture, tuning hyperparameters, increasing the amount of training data, or applying additional data augmentation techniques. Once changes have been made, the model is trained and evaluated again to determine how they affected its performance.

Changes to the model architecture may include adding or removing layers, increasing the number of neurons in the layers, applying more advanced convolutional operations, or using alternative activation functions. Adjusting the model's hyperparameters, such as training rate, regularization coefficients, and training packet size, can also significantly affect its performance.

Increasing the amount of training data allows the model to obtain more diverse examples and improve its generalization ability. Applying additional data augmentation techniques such as rotation, scaling, reflection, and illumination changes also helps to improve the generalization ability of the model.

Continuous model improvement is a key aspect in the development of computer vision systems. It enables the creation of more accurate and efficient models for various object detection tasks. By continuously analysing results, making adjustments, and experimenting with different approaches, significant improvements in model performance and object detection capability can be achieved [16–20].

6 Further Research

In addition to improving the current model, future research may include experimenting with other model architectures, developing new training and estimation methods, and investigating applications of computer vision systems in various fields such as medicine, automotive, and agroculture.

Experimentation with other model architectures may include exploring more complex deep neural networks, as well as developing lighter and more computationally efficient models that can run on mobile devices or in environments with limited computational resources.

The development of new training and evaluation methods can include the use of reinforcement learning techniques to improve the adaptability of models to new environments, as well as the development of new metrics for evaluating model performance that more accurately reflect the requirements of specific applications.

Further research plays an important role in advancing the field of computer vision by expanding our understanding of the capabilities and limitations of existing methods and models, as well as their application in various practical areas. This contributes to the development of more efficient and adaptive computer vision systems that can solve a variety of real-world problems [21].

7 Conclusion

In this study, we presented the process of training, validating and utilizing the YOLOv8 model for object detection in images. We used Ultralytics and Roboflow libraries to implement these processes, which allowed us to manage data and models efficiently.

The main steps and results of the study include:

Installation and environment setup: We installed the necessary libraries and prepared the environment to handle the data and models.

Model training: We trained the YOLOv8 model on the provided data, allowing the model to learn patterns and adapt to the specific object detection task.

Model validation: We validated the trained model to assess its quality and ability to generalize knowledge to new data.

Predicting objects in an image: We used the trained model to predict objects in a specific image, which allowed us to put the model into practice and evaluate its performance on real data.

These results demonstrate the effectiveness of using the YOLOv8 model with Ultralytics and Roboflow libraries to solve the problem of object detection in images. However, other advanced models such as Faster R-CNN, SSD, or EfficientDet could be considered to optimally solve a specific problem.

Continuous model improvement and further research in this area could lead to even more accurate and efficient computer vision systems. This could include experimenting with new model architectures, developing new training and evaluation methods, and investigating the application of computer vision systems in new application domains.

Overall, this study confirms the significance and promise of using deep learning methods to solve object detection problems in images, which is an important step in the development of the field of computer vision [22].

References

1. Tynchenko, V., et al.: Application of U-Net architecture neural network for segmen-tation of brain cell images stained with trypan blue. In: International Conference on High-Performance Computing Systems and Technologies in Scientific Research, Au-tomation of Control and Production. Springer Nature Switzerland. Cham (2023)
2. Shutaleva, A., et al.: Sustainability of inclusive education in schools and higher edu-cation: teachers and students with special educational needs. Sustainability **15**(4), 3011 (2023)
3. Tynchenko, V., et al.: Mathematical models for the design of GRID systems to solve resource-intensive problems. Mathematics **12**(2), 276 (2024)
4. Martyushev, N., et al.: Review of methods for improving the energy efficiency of electrified ground transport by optimizing battery consumption. Energies **16**(2), 729 (2023)
5. Suprun, E., et al.: The use of artificial intelligence to diagnose the disease. In: BIO Web of Conferences. vol. 84. EDP Sciences (2024)
6. Kukartsev, V., et al.: Study of the influence of the thermal capacity of the lining of acid melting furnaces on their efficiency. Metals **13**(2), 337 (2023)
7. Orlov, V., et al.: Development of a multifunctional cross-platform system for automa-tion of energy data and resource management. E3S Web of Conferences, vol. 460. EDP Sciences (2023)

8. Rezanov, V., et al.: Study of melting methods by electric resistance welding of rails. Metals **12**(12), 2135 (2022)
9. Kravtsov, K., et al.: Workflow automation and performance improvement based on PostgreSQL. In: E3S Web of Conferences. vol. 458. EDP Sciences (2023)
10. Chernykh, N., et al.: Comparative analysis of existing measures to reduce road accidents in Western Europe. In: 2023 22nd International Symposium INFOTEH-JAHORINA (INFOTEH). IEEE (2023)
11. Tynchenko, V., et al.: Effective energy management tools: inventory management and monitoring of energy consumption by personnel. In: E3S Web of Conferences, vol. 458. EDP Sciences (2023)
12. Kurashkin, S., et al.: Mathematical modelling of waveguide paths by electron-beam welding. Procedia Computer Science **200**, 83–90 (2022)
13. Semenova, E., et al.: Using UML to describe the development of software products us-ing an object approach. In: 2022 IEEE International IOT, Electronics and Mechatronics Conference (IEMTRONICS). IEEE (2022)
14. Tynchenko, V., et al.: Software for modeling brazing process of spacecraft elements from widely used alloys. In: 2022 21st International Symposium INFOTEH-JAHORINA (INFOTEH). IEEE (2022)
15. Volneikina, E., et al.: Simulation-dynamic modeling of supply chains based on big data. In: 2023 22nd International Symposium INFOTEH-JAHORINA (INFOTEH). IEEE (2023)
16. Rogova, D., et al.: Software system for modeling temperature distribution during the electron beam welding. In: 2022 IEEE International Conference on Design & Test of In-tegrated Micro & Nano-Systems (DTS). IEEE (2022)
17. Degtyareva, K., et al.: Automatic monitoring system designed to detect defects in PET preforms. E3S Web of Conferences, vol. 458. EDP Sciences (2023)
18. Boychuk, I., et al.: A methodological approach to the simulation of a ship's electric power system. Energies **16**(24), 8101 (2023)
19. Gantimurov, A., et al.: Investigation of the influence of geographical factors on soil suitability using a nonparametric controlled method of training and data analysis. In: E3S Web of Conferences, vol. 431. EDP Sciences (2023)
20. Malashin, I., et al.: Estimation and prediction of the polymers' physical characteristics using the machine learning models. Polymers **16**(1), 115 (2023)
21. Tynchenko, V., et al.: Using software to shape safety on the construction site. In: E3S Web of Conferences, vol. 458. EDP Sciences (2023)
22. Bukhtoyarov, V., et al.: Application of machine learning algorithms for refining pro-cesses in the framework of intelligent automation. SOCAR Proceedings **1**, 12–20 (2022)